

Dynamic Power Estimation

Using Cadence Voltus with Stylus Common UI

For a visual demonstration of the material in this document, please see the tutorial at:

- Part 1: <https://youtu.be/lz1pTSgo8HY>

- Part 2: https://youtu.be/Su-YhDa_tdo

1 Introduction

Power has long been one of the most important, if not the most important, metrics in VLSI design. That being said, it is often relegated to a secondary optimization target, and in most cases, it is not even measured or reported accurately. In this document, I will explain how to run a dynamic power estimation, based on the actual net toggling information extracted from a functional testbench. The flow described hereafter uses **Innovus** for place and route, **Xcelium** for gate-level simulation and **Voltus** for power reporting.

2 Power Consumption and Power Estimation

In general, power consumption in chip design can be divided into *dynamic power* and *leakage power*. Dynamic power is, of course, the power consumed during switching events, while leakage power is the static power of the design, consumed at all times. When considering a digital design, the standard cells and IPs are black boxes of the design, and so the dynamic power is further divided into switching power, which is the power consumed in the interconnect, and internal power, which is the power consumed internally inside the gates or IPs.

2.1 Switching Power

Switching power is the power consumed by charging and discharging interconnect capacitances that connect the pins of leaf cells (standard cells and IPs). Since this is a dynamic power component, the power is calculated as:

$$P = \frac{1}{2} A \cdot f \cdot C_L \cdot V^2 = \frac{1}{2} C_L \cdot V^2 D$$

where C_L is the output capacitive loading, V is the voltage, F is frequency, and A is the average switching activity. Sometimes the *transition density*, D , is used, where $D = AF$.

It is important to note that while capacitance is extracted from the design and the voltage is set by the user, the transition density is a factor of the input vectors and the propagation of the signals through the design. This is program dependent and is often hard to compute, which is why this entire tutorial is given. If we could just say that $A=0.2$ (a 20% activity factor), this whole tutorial would be unnecessary!

2.2 Internal Power

The *internal power* is the power consumed inside the cell during switching activities, i.e., the charging and discharging of interconnect capacitances that are not seen by the digital designer. Internal power is a strong function of the state transitions at the inputs (and sometimes outputs) of the cell/IP, and these values are described in the `.lib` file of the cell/IP. As with switching power, internal power is highly dependent on the

actual switching activity of each net in the design, and therefore, requires advanced tools and methodologies to perform power estimation.

2.3 Leakage Power

Leakage Power is the power consumed by devices when they are not switching. While this may seem trivial (i.e., just multiply the voltage times the leakage current...), the leakage current is also state dependent. For example, the leakage through a NAND gate is different for an input of $(A,B)=(0,0)$ vs $(A,B)=(0,1)$. Therefore, the internal state of the design, or at least the probability (or duty cycle) of each net is required for a good leakage power estimation.

3 Types of Power Calculation in Voltus

Note – if this is your first time reading this, you may not understand it. Skip to Section 4 and come back later 😊.

3.1 Vector-based Average Power Calculation

The vector-driven approach uses the **VCD** or **TCF** output of a logic simulator to obtain the number of transitions for each net. Gate-level VCD or TCF with good functional coverage are required for accurate power calculation results. You can use **VCD** or **TCF** information to calculate accurate power consumption figures, if the following conditions apply:

- Gate-level simulation is possible at the full-chip level.
- Gate-level simulation provides sufficient functional coverage for the design.
- The vectors include those that cause the highest power consumption.

The power engine also takes clock definition from **VCD** or **TCF** and gives higher priority to clock definition from **VCD** or **TCF** if there is discrepancy between clock frequency in **SDC** or **TWF** and **VCD** or **TCF**.

3.2 Propagation-based Average Power Calculation

The power engine calculates the switching probability, as well as static state probability, of each net in the design. The propagation-based approach is vector-independent and provides coverage for all nets in a design. However, the accuracy depends on good starting values, that is, information about the switching probabilities at the primary inputs in a design. Simple examples are **clock** and **reset** or **enable** inputs. Obtaining an accurate prediction without information about the switching probabilities of these special inputs is difficult, and in most cases an inaccurate prediction causes an over estimation of the power consumption. With the power engine you can use either the `set_default_switching_activity` command to specify the average activity on sequential cells or use **RTL**, **VCD**, or **TCF** for seeding activity at sequential cells.

4 Overview of Power Analysis Flow

I have found that students have a hard time understanding the *flow* of power estimation, so please read this section carefully before starting to try and actually run the flow.

Vector-based power analysis takes the activity data from a logic (functional) simulation and calculates the power consumption of the design based on this activity. This is a non-trivial task for several reasons. First, to get a good power estimation, we should be using a gate-level design, and even better, a post-layout design, so that we have

the exact timing and parasitics information. Second, it takes different tools to run this – one for implementing the design, one for running the logic simulation, and one for calculating the power. Moving between various tools is often confusing and can be challenging. Finally, it is complex, since running a post-layout simulation on a large design is time-consuming to the point of often not being feasible. But you can also run on only part of the design or run a short simulation to try and overcome the size/runtime challenge.

The flow of vector-based power estimation is as follows:

1. **Create a functional testbench** for the operation you would like to test for power consumption. For example, if you were designing a standard cell based memory, you would write a testbench for performing write and read operations. Verify that your *device-under-test (DUT)* operates properly.
2. **Implement your design** to get a gate-level netlist. This can be through synthesis (e.g., with **Genus**) or can include a full place and route procedure to provide a *post-layout netlist*.
3. **Extract the timing information** from your design. The static timing analysis tool within the synthesis (e.g., **Genus**) or place and route (e.g., **Innovus**) tool can extract this information. It is generally exported as a *standard delay format (.sdf)* file.
4. **Run a gate-level simulation with timing information**. This means that you run *the same testbench* as in stage #1, but replace your RTL based DUT with the gate-level netlist produced in stage #2. In addition, you need to import the timing information (*.sdf* file) to create delays in the logic simulation.
5. **Export the activity information** from the gate-level simulation. The activity information is just a dump of all the toggling of nets during the simulation. This is commonly done with a *value change dump (.vcd)* file, but can also be done with the more compact *toggle count format (.tcf)*.
6. **Calculate the power consumption** using a power calculation engine, such as **Voltus**. This requires loading the design (netlist and parasitics) into the power tool, reading in the activity information (*.vcd/.tcf*) and calculating the power.
7. **Write out reports and debug**. Debugging is very important – make sure you *agree* with the reports and if not, figure out what went wrong. Do this by perform basic *sanity checks* that show that what you expected is what you got. For example, if we were performing write-power analysis to our memory from above, we would see power on the write decoder, in the registers and on the write clock. If we were performing read-power analysis, we should see that the read decoder and mux are gobbling up power, but the registers and write decoder are not.

To quickly summarize this within a Cadence-based flow, these are the operations you will be carrying out:

1. Write a testbench and verify that it runs correctly on your RTL design using **Xcelium**.
2. Synthesize with **Genus** and optionally place and route with **Innovus**.
3. Save your gate-level netlist (write_netlist) and the timing information (write_sdf).
4. Run a post-layout simulation with .sdf backannotation using **Xcelium**.
5. Export the activity information as a *.vcd* or *.tcf* file.
6. Import your design to **Voltus** and read the activity information (read_activity_file)
7. Calculate the power in **Voltus** (report_power) and go over the reports.

5 Step-by-Step Instructions

Who ever said we don't hand feed you? 😊

5.1 Designing, Simulating, Implementing

So, I'm assuming that you already know how to do these (stages 1&2 above). If not, go back and learn how to design, simulate, synthesize, place and route!

5.2 Saving design and timing information

Okay, so now I am assuming that you are going for a full-blown post-layout power estimation flow (maybe I'll add post-synthesis instructions later). So, we're inside **Innovus** and we just finished post-route optimization with no DRC or connectivity violations.

Just to point out, this is not a requirement. You can have both DRC (`check_drc`) and connectivity (`check_connectivity`) violations and the flow will still run. You don't even have to be post-route – you can do this after placement, CTS, whatever. But it will only be true and accurate, once your design is routed and clean.

Now just do the following steps.

5.2.1 Save your post-layout netlist

Well, this is pretty simple:

```
write_netlist <design_name>.v
```

This saved your gate-level netlist under the name `<design_name>.v`

5.2.2 Save your timing information (.sdf)

A standard delay format (`.sdf`) file is a file that contains timing information about your design. It is divided into interconnect (nets) and gates. For each condition of the inputs of a gate, it provides timing information for rising and falling arcs. Each delay is reported as a tuple (three values), representing (`worst_case` : `typical` : `best_case`) corners.

```
write_sdf <design_name>.sdf
```

This saved your `.sdf` file under the name `<design_name>.sdf`

Note that the `write_sdf` command has many options. If you are having trouble with the delay backannotation in your simulation, check the options and look at the `.sdf` file itself to see if this is the root of your problem.

5.2.3 Save your Innovus database

By saving your **Innovus** database, you can easily reload all the design, extraction and timing information back into **Voltus** (or run **Voltus** integrated in **Innovus**). Anyway, it's a really good idea to always save your work...

```
write_db <design_name>.db
```

This just saved your design in a directory called `<design_name>.db`

5.3 Running Gate-Level Simulation with SDF back-annotation

So, we can now leave **Innovus** and move on to the next step (step #4 in the flow), which we do in a logic simulator, such as **Xcelium** or **ModelSim**. The instructions here are for **Xcelium**. Remember that our goal is to run a gate-level simulation on the design, while taking into account the timing information (**.sdf**) that we extracted from **Innovus**. The simulation should dump an activity file (**.vcd** or **.tcf**), which we will analyze with **Voltus**. But first, we have to prepare the testbench for gate-level simulation with **.sdf** back-annotation.

5.3.1 Preparing the back-annotation testbench

To clarify the terminology, “back-annotation” in this context means to take the timing information from the **.sdf** and apply it to the DUT in the testbench, i.e., to “annotate” the simulated design with timing information. The approach that Cadence takes for this step is to actually write the commands inside the **Verilog** testbench (yes, I know it’s weird...).

So, basically, what you have to do is add an **initial** block in the testbench that tells the simulator to annotate the design with the information in the **.sdf** file. This is done using the **\$sdf_annotate** command.

```
initial
begin
    $sdf_annotate(<design_name>.sdf,<DUT>,, "sdf.log" , "MAXIMUM");
end
```

The parameters given to the **\$sdf_annotate** command are important:

1. **Name** of the **.sdf** file
2. The “**scope**” of the **.sdf** file. This is the name of the instance inside the testbench that the **.sdf** file relates to. In most cases this will be the instantiation name of the **Verilog** netlist inside the testbench, but you can actually take several **.sdf** files and map them to several instantiations inside the testbench.

Note that the scope is the first place to check if your annotation is not working!
Remember that I told you so...

3. Leave it empty – there are two *commas* there on purpose!
4. Name of the **logfile**.
5. The **corner** to take from the **.sdf** file (remember, there is a tuple of (**worst:typical:best**))

To find out more about the **\$sdf_annotate** command and **.sdf** Back-annotation, in general, refer to the “SDF Annotator Guide”, available on Cadence Support.

5.3.2 Define the Gate-Level Simulation

To run the gate-level simulation, you need to read the gate-level netlist instead of the RTL of the DUT. You can do this in various ways, but let's assume that you are using a `.f` file to list all the files to read into the simulator, so your original `.f` file would look something like this:

```
// my_testbench.f
//-----
defines.v
tb.v
first_file.v
second_file.v
third_file.v
-v <path_to_digital_libraries>/Verilog/<sc_library_name>.v
```

Instead, you need to read the gate-level netlist. Also, you must read the Verilog behavioral descriptions of the standard cells (which are now instantiated in your design). So, your `.f` file for backannotation will probably look like this:

```
// my_backannotation_testbench.f
//-----
defines.v
tb_backannotation.v
<design_name>.v
-v <path_to_digital_libraries>/Verilog/<sc_library_name>.v
```

In other words, you only read in the gatelevel netlist, since you have already compiled and synthesized all the other RTL files, and therefore, no longer need them.

Note that it is really good practice to use the *exact same* testbench for gatelevel and RTL simulation by manipulating ``define` and ``ifndef` commands inside the Verilog for defining the `.sdf` backannotation and any other required changes.

5.3.3 The xrun command line for backannotation

So now that we have the testbench and `.f` files set up, we can run our simulation:

```
xrun -f <f_file_name>.f +sdf_verbose -sdfstats sdf_stats.txt
```

The **Xcelium** simulator has many command line options, of course, but specifically:

- `-f` : Points to a `.f` file to list all the **Verilog** files to read.
- `+sdf_verbose` : writes out the sdf logfile .
- `-sdfstats` : writes out the sdf statistics file.
- `-sv` : add this if you have System Verilog code
- `-gui` : add this if you want to open the **SimVision** waveform viewer.
- `-debug` : add this if you want to save a database for debugging (replace +rw +access)
- `-input <script file>` : this runs a **tcl** script after loading the simulator. This is actually an essential step, as will be explained below.

Again, check that functionally, everything is working, as it did with the RTL simulation (otherwise, go ahead and debug...)

5.3.4 Checking that the SDF Backannotation worked

Now open the log files that the SDF annotation outputs and see that everything looks okay:

- **sdf.log** : It should show numbers annotated on every instance of the design. If everything is 0.000 then you should be worried...
- **sdf_stats.txt** : This file shows a list of unannotated paths. It shouldn't be too long. Actually, I'm not entirely sure why it has any entries at all, but certain things do not get annotated by **Innovus**, it seems.

A very important thing is to check delays in the waveforms. If zero delays are applied, like in RTL simulation, then something is wrong. If you have a delay through each logic gate, that is a good sanity check to assume that everything is okay.



Figure 1: SimVision waveform with delays due to SDF back-annotation

5.3.5 Dumping the activity file

The last thing to take care of within the **Xcelium** environment is exporting an activity file.

There are two formats of activity files that can be used for power estimation:

- **Value Change Dump (.vcd)**: This is a file that shows changes of a signal (net) at each timestep (usually the resolution of 1ps) in the functional simulation. It is clear that for a big design (i.e., lots of nets), the file size grows very quickly with the simulation time (i.e., lots of time steps).
- **Toggle Count Format (.tcf)**: As a means to condense the activity file (since .vcd files are so large), Cadence came up with an alternative format that just counts the number of changes that occur at the output of each gate. For power estimation, this is sufficient, since all that is needed is the number of times a capacitor was charged or discharged within a given time period. In addition, the *duty cycle* of a signal is recorded (i.e., what percentage of the time it was '1' or '0') for calculating the leakage power.

The method for dumping a **.vcd** or **.tcf** file is a bit different. For a **.vcd** file, you tell **SimVision** to create a **.vcd** database and then to **probe** the signals you want before running the simulation:

```
database -open <design>.vcd -vcd
probe -create <DUT> -vcd -depth 9 -all -database <design>.vcd
```

Note that the **-depth** parameter tells the probe how deep into the hierarchy to go. So, if you have a flat gate-level design, **-depth 2** should be enough. But you won't get 100% coverage if your design has more hierarchies.

For a **.tcf** file, on the other hand, you first open the file, then run the simulation and then close the file:

```
dumptcf -output <design>.tcf -scope <TB>.<DUT>
run -time <simulation run time>
dumptcf -end
```

Note that while the difference pointed out above seems almost trivial, it does present a different approach. The .vcd, for instance, has the notion of time, so if you know you performed the interesting operation between time a and time b , you can just read the part of the .vcd file that occurred between these two timesteps. The .tcf file, on the other hand just says how many toggles there were while the file was being written. Therefore, you have to explicitly write out a .tcf file from the simulator during the relevant time period that you want to evaluate power for.

5.3.6 Automating the simulation with a .tcl command file

In order to automate the back-annotation run, including the creation of the dump file, it is highly recommended to start **Xcelium** with a .tcl command file:

```
%> xrun -f <f_file_name>.f -input <command_file>.tcl
```

With the complex approach of the **xrun** command, I will just note that the commands are sent to **NCSim** (one of the executables in the **Xcelium** suite). If you want to run a command in a different tool (e.g., **SimVision**), you have to start the command with the tool name. See the example .tcl file below to better understand:

```
# my_simulation_commands.tcl
simvision scope set <TB>
set w [simvision waveform new]
simvision waveform add -using "Waveform 1" -signals <SCOPE>
database -open <design>.vcd -vcd
probe -create <DUT> -vcd -depth 9 -all -database <design>.vcd
dumptcf -output <design>.tcf -scope <TB>.<DUT> -overwrite -verbose
run -time <simulation time>
dumptcf -end
simvision exit
```

The above set of commands opens up a waveform window in **SimVision**, adds the relevant signals (the “scope” of the simulation, which would be “**TB.DUT**” for a testbench called **TB** and an instantiation of a toplevel block called **DUT**) to the waveform, and a .vcd database called <design>.vcd is opened. All signals are probed (dumped to the .vcd database) and in addition, a .tcf file called <design>.tcf counts their toggles. The simulation is run for <simulation time> and the .tcf file is closed, before the tool is closed and exits.

5.4 Estimating Power with Voltus

The final stage of the procedure is to calculate the power within **Voltus**. The **Voltus** tool is an extensive set of commands for **power estimation**, **IR drop** calculation and more. We are going to use it for a very simple operation for now. First, let’s start the tool:

```
%> voltus -stylus
```

Note that **Voltus** can be run within the shell of **Innovus**. In other words, all the commands that will be presented below can be run within a standalone **Voltus** shell or as a continuation of our place and route run without leaving **Innovus**.

5.4.1 Importing an Innovus Database

To get started, you need to import the design you implemented in **Innovus** or **Genus** into **Voltus** (or back into **Innovus** for running **Voltus** within an **Innovus** shell). The easiest way to do this is to restore the **Innovus** database that we saved in Section 5.2.3.

```
read_db <design>.db
```

It is probably insufficient to *just* restore the design. You may need to reload your variable and library definitions, as well as redefining the global net connections. For example:

```
source <design>.defines
source libraries.tcl
connect_global_net GND -pin gnd -all -verbose
connect_global_net VDD -pin vdd -all -verbose
```

5.4.2 Setting Voltus to vector-based power analysis

As previously mentioned, **Voltus** has various modes. We need to set it to the vector-based power analysis mode, which in **Voltus** is (for various reasons) called “**static**” mode:

```
set_db power_method static
set_db power_report_missing_nets true
set_db power_view wc_analysis_view
```

the **power_view** option should choose the MMMC analysis view that you want to run the power estimation for.

5.4.3 Reading an activity file

So now we can read in the activity file that we exported from **Xcelium**. Let's start with the .tcf file:

```
read_activity_file -reset
read_activity_file -format TCF -scope <DUT> <design>.tcf
```

If, instead, you choose a **.vcd** flow, then just change the command to read in a **.vcd** file:

```
read_activity_file -format VCD -scope <TB>.<DUT> <design>.vcd
```

Actually, this probably isn't enough in the case of a **.vcd** file. The VCD probably has more information that you actually want. For example, let's say the first 1000ns were used to reset the simulation and we don't want to take this into account in our power estimations. Maybe the relevant time we want to account for is only 500ns, so we would need to start the activity at 1000ns and end it at 1500ns of the simulation. Therefore we would add the **-start** and **-end** options:

```
read_activity_file -format VCD -start 1000 -end 500 \
-scope <TB>.<DUT> <design>.vcd
```

WARNING – the **-scope** definition is really problematic. If you follow *exactly* what I wrote here, this should work, but if you deviate a bit, you will have annotation problems. I spent hours trying to get this to work properly. Don't be surprised if you do too...

5.4.4 Calculating the power consumption

Okay, we are finally at the point that we've worked so hard to get to. So here it is, just use the **report_power** command:

```
report_power -rail_analysis_format VS -out_file power.rpt
```

This will create a file called “**power.rpt**” with your power report!

5.5 Understanding and Debugging the power report

We now have our results. But, as a VLSI engineer, you know that nothing can go right on the first try. So how do we check that our results are correct? How do we know what the actual power consumption is? What do we do with these numbers?

5.5.1 Checking design annotation coverage

Let’s start by opening the power report. Pretty much at the top of the file, there will be a report on the design annotation coverage. In other words, was the tool able to map the signals in the **.tcf/.vcd** file to the signals in the **Innovus/Voltus** database:

```
* Switching Activity File used:
*   ../../TB/backannotation/simvisionTcl.vcd
*       Vcd Window used(Start Time, Stop Time): (1162,2160)
*       Vcd Scale Factor: 1
*       Design annotation coverage: 0/983 = 0%
```

This is VERY BAD! It means you didn’t heed to my warning about the **-scope** before and now you have to start figuring out why you are having annotation problems.

What you should be seeing is this:

```
* Switching Activity File used:
*   ../../export/read.tcf
*       Design annotation coverage: 983/983 = 100%
```

The above is from a **.tcf** file. For a **.vcd** file, it would be very similar:

```
* Switching Activity File used:
*   ../../export/tb_scm65.vcd
*       Vcd Window used(Start Time, Stop Time): (1,1161)
*       Vcd Scale Factor: 1
* *       Design annotation coverage: 983/983 = 100%
```

Note that in the **Voltus/Innovus** log file, you will find even more annotation statistics after running the **read_power** command. They will tell you what percentage of the design actually appears in the **.vcd/.tcf** files. While this should be as close to 100% as possible, it is not a must (in fact, it usually will be more like 75% for various reasons).

5.5.2 The Power Report

The actual power report appears right below the header information in the file. This part shows a per-instance report of the power consumption:

Cell	Internal Power	Switching Power	Total Power	Leakage Power	Cell Name
GWCLK_gate	0.0003628	0.006114	0.006478	6.955e-07	PREICG_X0P5B_A12TR
GDINCLK_gate	0.000399	0.003793	0.004192	6.955e-07	PREICG_X0P5B_A12TR
bitslice_8/GDIN_reg	0.0009925	0.00139	0.002385	2.763e-06	DFFQ_X2M_A12TR

As you can see by the table header, the report is divided into six columns:

- **Cell:** The name of the instance in the design

- **Internal Power:** The internal power consumption of the cell
- **Switching Power:** The switching power consumption of the cell
- **Total Power:** The total power consumption of the cell (**Internal+Switching+Leakage**)
- **Leakage Power:** The leakage power of the cell
- **Cell Name:** The library cell of the instance (the standard cell or IP name)

The cells appear in descending order, sorted by total power consumption. The units are by default in milliwatts (as written in the file header).

At the end of the file, you will get a summary of the power consumption:

```
-----
Total ( 955 of 1009 ) 0.03582 0.0402 0.07705 0.001025
Total Capacitance 7.478e-12 F
Power Density *** No Die Area ***
-----
```

As you can see, this reports the sum of all the internal, switching, total, and leakage power. In addition, it tells us how many cells (955) out of the total number of cells in the design (1009) appear in the report. It also tells us the total capacitance found in the design (7.478 pF). Power density will be reported if you define the die area.

5.5.3 Sanity check on your power report

Please, do me a favor, and don't take your report as the ground truth. Make sure that it makes sense. You know your design and your testbench, so make sure that the cells that are consuming a lot of power are the actual cells that *should* be consuming power.

For example, in the reports from the example above, the testbench was doing write operations to the memory. Accordingly, the biggest power consumers are two clock gates on the write clock, which makes sense, since they are constantly toggling. The next gate is a write data register that samples new data every clock cycle. If we go further down into the report, we will see that the write decoder consumes quite a bit of power, while the read decoder hardly consumes any. The memory latches also consume power, as they are being updated, but in the read testbench power report, the memory latches do not toggle at all, and therefore, appear towards the bottom of the report.

The scenario I explained above *makes total sense*. This is what I would expect to find in my power report. That is a *sanity check*. Now, I can have better trust in the numbers. There can be many other things that went wrong, but now I have some confidence about this. If I would have seen the read decoder consuming power in the write testbench, I bet you that either the annotation coverage was low (which means that Voltus propagates a default activity factor of about 20% to each gate), or I was looking at the wrong time period in my `.vcd` (or `.tcf`) file.

Remember, *sanity check*!

Don't leave home without it!

This document was provided to you courtesy of Prof. Adam Teman of the EnICS Labs Institute in the Alexander Kofkin Faculty of Engineering at Bar-Ilan University. The content was entirely written and compiled by Prof. Teman (without the help of AI tools), based on knowledge and non-proprietary data. You can find all of Prof. Teman's lectures and materials at the EnICS Labs website: <https://enicslabs.com/education/>

For any comments or enquiries, Prof. Teman can be reached at adam.teman@biu.ac.il